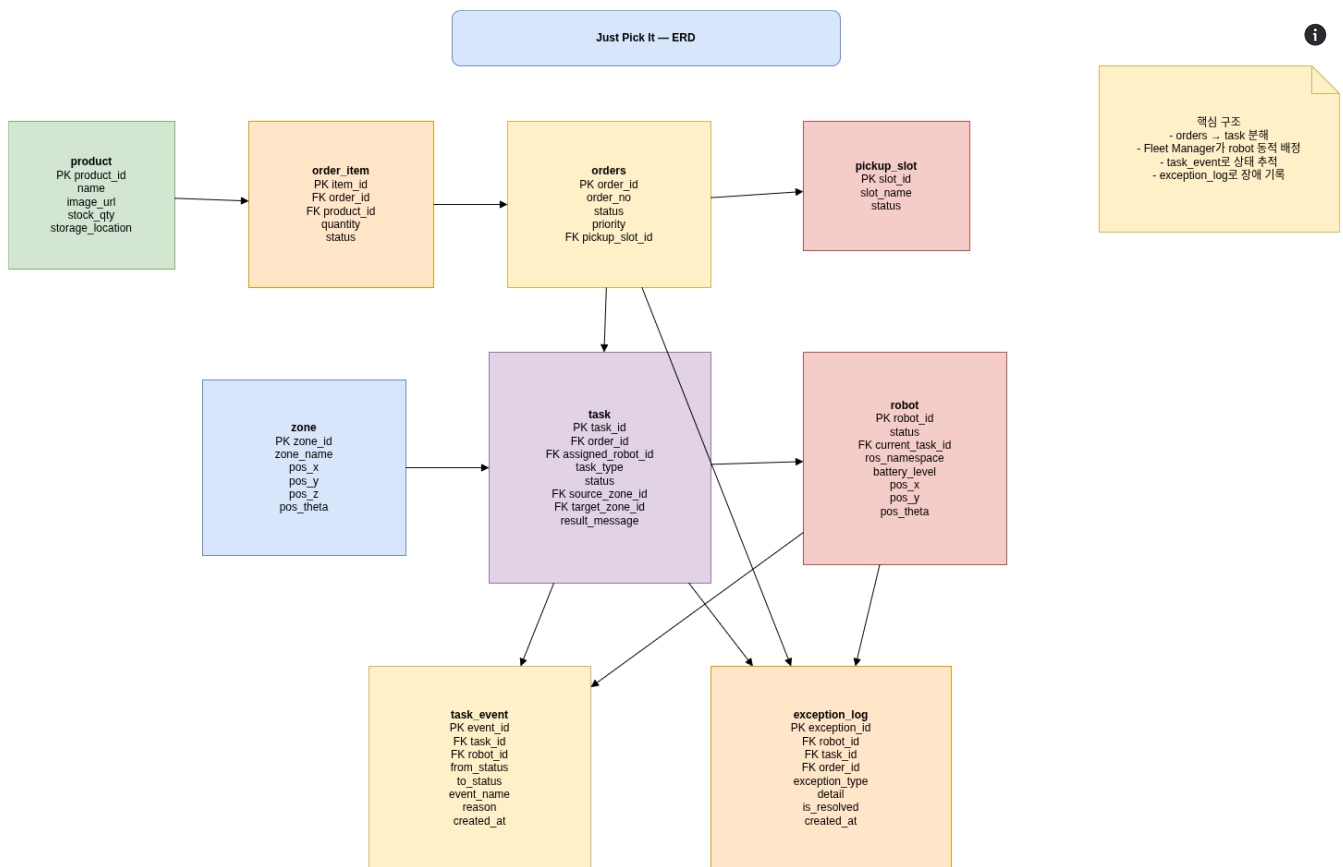


Data Structure

1. 전체 데이터 흐름

```
1 고객 주문 생성
2   → orders 생성
3   → order_item 생성
4   → task 생성
5   → 각 Robot Control Node 또는 Control Bridge가 자기 로봇의 task 조회
6   → Robot Control Node가 task 시작/완료/실패 상태 보고
7   → robot 상태 갱신
8   → task_event 기록
9   → 검수 시작 시 pickup_slot RESERVED 배정
10  → 하차 완료 후 pickup_slot OCCUPIED 변경
11  → 고객 수령 완료
12  → 예외 발생 시 exception_log 기록
```

2. ERD 개요



핵심은 고객 주문을 로봇 작업 단위인 task로 분해하고, 각 Robot Control Node 또는 Control Bridge가 자기 로봇에 배정된 task를 조회해 수행 상태를 Control Server에 보고하는 구조이다.

3. 테이블 역할 요약

테이블	역할	사용 위치
-----	----	-------

zone	로봇이 이동하거나 작업하는 주요 위치 정보	Robot Control Node, AMR 목적지, Cobot 작업 위치
product	상품 정보와 재고	고객 상품 목록, 관리자 재고 관리
pickup_slot	픽업 슬롯 관리	픽업존, 고객 수령
orders	주문 1건의 대표 상태	고객 주문 상태, 관리자 주문 현황
order_item	주문 안에 포함된 상품 목록	Cobot 선별 / 검수
robot	로봇 4대의 현재 상태	관리자 관제, Robot Control Node
task	로봇에게 줄 작업 단위	작업 큐, 자동 작업 배정
task_event	작업 진행 이력	관리자 작업 타임라인
exception_log	예외 상황 기록 및 관리	장애 분석, 관리자 대시보드

4. ENUM 타입 정의

```

1 CREATE TYPE order_status AS ENUM (
2     'ORDER_RECEIVED', -- 주문 접수
3     'ORDER_WAIT',    -- 주문 대기
4     'SORTING',       -- 상품 선별 중
5     'DELIVERING',    -- 상품 운반 중
6     'INSPECTING',    -- 상품 검수 중
7     'PICKUP_READY',  -- 픽업 준비 완료
8     'COMPLETED',    -- 수령 완료
9     'ERROR'          -- 예외 발생
10 );
11
12 CREATE TYPE order_item_status AS ENUM (
13     'WAITING', -- 작업 대기 중
14     'SORTED',  -- Cobot1이 AMR에 선별 상차 완료
15     'INSPECTED', -- Cobot2 상품 검수 완료
16     'MISSING',  -- 검수 결과 상품 누락
17     'EXCESS',   -- 검수 결과 상품 초과
18     'MISMATCH'  -- 검수 결과 불일치
19 );
20
21 CREATE TYPE pickup_slot_status AS ENUM (
22     'EMPTY', -- 비어 있음
23     'RESERVED', -- 주문에 배정됨
24     'OCCUPIED', -- 상품 있음 / 고객 픽업 대기

```

```

25     'BLOCKED'          -- 사용 불가
26 );
27
28 CREATE TYPE robot_status AS ENUM (
29     'IDLE',             -- 대기 중
30     'MOVING',           -- 이동 중
31     'WAITING',          -- Cobot 작업 대기 중
32     'STANDBY',          -- AMR 상하차 대기 중
33     'SORTING',          -- Cobot 선별 작업 수행 중
34     'LOADING',          -- 상품 AMR 상차 중
35     'PARKING',          -- 주차 중
36     'INSPECTING',       -- Cobot 검수 작업 수행 중
37     'UNLOADING',        -- AMR 상품 하차 중
38     'PATROLLING',       -- 순찰 중
39     'CHARGING',          -- 충전 중
40     'RETURNING',        -- 충전구역 복귀 중
41     'DOCKING',          -- 도킹 중 (조심해서 사용 충전 도크에서만 사용 해야
    함)
42     'EMERGENCY_STOP',   -- 긴급 정지 상태
43     'ERROR',            -- 로봇 오류 상태
44     'OFFLINE'           -- 연결 끊김
45 );
46
47 CREATE TYPE task_type AS ENUM (
48     'STANDBY_LOAD',      -- 상차 대기 (AMR1 또는 AMR2)
49     'STANDBY_UNLOAD',    -- 하차 대기 (AMR1 또는 AMR2)
50     'LOAD',              -- 상품 상차 (AMR1 또는 AMR2)
51     'UNLOAD',            -- 상품 하차 (AMR1 또는 AMR2)
52     'SORTING',           -- 상품 선별 및 상차 (Cobot1)
53     'INSPECTION',        -- 상품 검수 및 하차 (Cobot2)
54     'PATROL',            -- 순찰 작업 (AMR1 또는 AMR2)
55     'RETURN_HOME',       -- 대기구역 복귀 중
56     'CHARGE',            -- 복귀 완료하여 충전 (AMR)
57 );
58
59 CREATE TYPE task_status AS ENUM (
60     'QUEUED',            -- 작업 큐 대기 중
61     'ASSIGNED',          -- 특정 로봇에 할당됨
62     'RUNNING',           -- 현재 수행 중
63     'PAUSED',            -- 긴급 정지 등으로 일시 중단
64     'SUCCESS',           -- 작업 성공 완료
65     'FAILED',            -- 작업 실패
66     'CANCELLED'         -- 작업 취소
67 );
68
69 CREATE TYPE exception_type AS ENUM (
70     'OBSTACLE_DETECTED', -- 장애물 감지
71     'LOW_BATTERY',       -- 배터리 부족
72     'NAVIGATION_FAILED', -- 경로 탐색 또는 주행 실패
73     'HARDWARE_ERROR',    -- 하드웨어 오류
74     'TIMEOUT',           -- 작업 시간 초과
75     'SORTING_FAIL',      -- 상품 선별 실패
76     'INSPECTION_FAIL',   -- 검수 실패
77     'HUMAN_DETECTED',    -- 사람 감지
78     'SYSTEM_ERROR',      -- 시스템 오류
79     'FIRE_DETECTED'      -- 화재 감지
80 );

```

5. 테이블 정의

5-1. zone

zone은 로봇이 이동하거나 작업하는 주요 위치를 저장한다.

AMR은 `x`, `y`, `theta` 를 기준으로 이동하고, Cobot 작업 위치나 상품 위치는 `z` 값을 함께 사용할 수 있다.

아래 좌표값은 예시이며 실제 맵 제작 후 수정한다.

```
1 CREATE TABLE zone (  
2     zone_id      SERIAL PRIMARY KEY,  
3     zone_name    VARCHAR(50) NOT NULL,  
4     pos_x        FLOAT NOT NULL,  
5     pos_y        FLOAT NOT NULL,  
6     pos_z        FLOAT NOT NULL,  
7     pos_theta    FLOAT  
8 );  
9  
10 INSERT INTO zone (zone_name, pos_x, pos_y, pos_z, pos_theta) VALUES  
11     ('STANDBY_LOADING_ZONE', 0.0, 0.0, 0.0, 0.0),  
12     ('LOADING_ZONE',        0.0, 0.0, 0.0, 0.0),  
13     ('STANDBY_UNLOADING_ZONE', 0.0, 0.0, 0.0, 0.0),  
14     ('UNLOADING_ZONE',      0.0, 0.0, 0.0, 0.0),  
15     ('CHARGING_DOCK',        0.0, 0.0, 0.0, 0.0),  
16     ('PRODUCT_ZONE',        0.0, 0.0, 0.0, 0.0);
```

5-2. product

product는 상품 정보와 재고를 저장한다.

고객 UI에서는 상품명, 이미지, 재고를 보여주고, 관리자 UI에서는 재고를 수정한다.

가격 정보는 본 프로젝트 범위에서 제외한다.

```
1 CREATE TABLE product (  
2     product_id    SERIAL PRIMARY KEY,  
3     name          VARCHAR(100) NOT NULL,  
4     image_url     TEXT,  
5     stock_qty     INT NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),  
6     storage_location VARCHAR(50) NOT NULL  
7 );
```

5-3. pickup_slot

pickup_slot은 고객이 상품을 가져갈 픽업 칸을 관리한다.

주문과의 연결은 `orders.pickup_slot_id` 로만 관리한다.

```
1 CREATE TABLE pickup_slot (  
2     slot_id      SERIAL PRIMARY KEY,  
3     slot_name    VARCHAR(50),  
4     status       pickup_slot_status NOT NULL DEFAULT 'EMPTY'  
5 );  
6  
7 INSERT INTO pickup_slot (slot_name, status) VALUES  
8     ('Pickup_slot_1', 'EMPTY'),  
9     ('Pickup_slot_2', 'EMPTY');
```

5-4. orders

orders는 주문 1건의 대표 정보를 저장한다.

고객 입장에서는 주문 진행 상태를 확인하는 기준이고, 관리자 입장에서는 전체 주문 현황을 보는 기준이다.

```
1 CREATE TABLE orders (  
2   order_id      SERIAL PRIMARY KEY,  
3   order_no      VARCHAR(30) UNIQUE,  
4   status        order_status NOT NULL DEFAULT 'ORDER_RECEIVED',  
5   priority      INT NOT NULL DEFAULT 1,  
6   pickup_slot_id INT REFERENCES pickup_slot(slot_id)  
7 );
```

5-5. order_item

order_item은 주문 안에 들어 있는 상품 목록을 저장한다.

예를 들어 고객이 콜라 2개, 과자 1개를 주문하면 orders는 1개 생성되고, order_item은 2개 생성된다.

```
1 CREATE TABLE order_item (  
2   item_id      SERIAL PRIMARY KEY,  
3   order_id     INT NOT NULL REFERENCES orders(order_id) ON DELETE  
4   CASCADE,  
5   product_id  INT NOT NULL REFERENCES product(product_id),  
6   quantity    INT NOT NULL CHECK (quantity > 0),  
7   status      order_item_status NOT NULL DEFAULT 'WAITING'  
8 );
```

5-6. robot

AMR1과 AMR2는 상황에 따라 DELIVERY 또는 PATROL task를 수행할 수 있으며, 각 AMR 담당 노드는 자기 로봇에 배정된 task를 조회해 실행한다.

```
1 CREATE TABLE robot (  
2   robot_id      VARCHAR(30) PRIMARY KEY,  
3   status        robot_status NOT NULL DEFAULT 'IDLE',  
4   current_task_id INT,  
5   ros_namespace VARCHAR(50),  
6   battery_level INT CHECK (battery_level BETWEEN 0 AND 100),  
7   pos_x         FLOAT,  
8   pos_y         FLOAT,  
9   pos_theta     FLOAT  
10 );  
11  
12 INSERT INTO robot (robot_id, status, ros_namespace, battery_level)  
13 VALUES  
14 ('COBOT1', 'IDLE', '/cobot1', NULL),  
15 ('COBOT2', 'IDLE', '/cobot2', NULL),  
16 ('AMR1', 'IDLE', '/amr1', 100),  
17 ('AMR2', 'IDLE', '/amr2', 100);
```

5-7. task

task는 로봇이 실제로 수행할 작업 단위다.

orders가 고객 관점의 주문이라면, task는 로봇 관점의 실행 명령이다.

```
1 CREATE TABLE task (  
2     task_id          SERIAL PRIMARY KEY,  
3     order_id         INT REFERENCES orders(order_id) ON DELETE  
4     CASCADE,  
5     assigned_robot_id VARCHAR(30) REFERENCES robot(robot_id),  
6     task_type        task_type NOT NULL,  
7     status           task_status NOT NULL DEFAULT 'QUEUED',  
8     source_zone_id   INT REFERENCES zone(zone_id),  
9     target_zone_id   INT REFERENCES zone(zone_id),  
10    result_message    TEXT  
11 );  
12 ALTER TABLE robot  
13     ADD CONSTRAINT fk_robot_current_task  
14     FOREIGN KEY (current_task_id) REFERENCES task(task_id);
```

5-8. task_event

task_event는 작업 상태 변화와 주요 이벤트를 기록한다.

예를 들어 선별상차 작업 하나 안에서도 작업 할당, 작업 시작, 상품 상차 완료, 작업 성공 같은 이벤트가 생길 수 있다.

```
1 CREATE TABLE task_event (  
2     event_id         SERIAL PRIMARY KEY,  
3     task_id          INT NOT NULL REFERENCES task(task_id) ON DELETE  
4     CASCADE,  
5     robot_id         VARCHAR(30) REFERENCES robot(robot_id),  
6     from_status      task_status,  
7     to_status        task_status NOT NULL,  
8     event_name       VARCHAR(50),  
9     reason           TEXT,  
10    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()  
11 );
```

5-9. exception_log

exception_log는 로봇이나 시스템에서 발생한 문제를 기록한다.

관리자 UI에서는 exception_log를 알림 목록처럼 표시한다.

is_resolved 는 관리자가 해당 예외를 처리했는지 표시하는 값이다.

```
1 CREATE TABLE exception_log (  
2     exception_id     SERIAL PRIMARY KEY,  
3     robot_id         VARCHAR(30) REFERENCES robot(robot_id),  
4     task_id          INT REFERENCES task(task_id),  
5     order_id         INT REFERENCES orders(order_id),  
6     exception_type   exception_type NOT NULL,  
7     detail           TEXT,  
8     is_resolved      BOOLEAN NOT NULL DEFAULT FALSE,  
9     created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()  
10 );
```

6. 인덱스 설계

관리자 화면과 Robot Control Node에서 자주 조회할 컬럼

```
1 CREATE INDEX idx_orders_status
2 ON orders(status);
3
4 CREATE INDEX idx_order_item_order_id
5 ON order_item(order_id);
6
7 CREATE INDEX idx_robot_status
8 ON robot(status);
9
10 CREATE INDEX idx_robot_current_task_id
11 ON robot(current_task_id);
12
13 CREATE INDEX idx_task_order_id
14 ON task(order_id);
15
16 CREATE INDEX idx_task_assigned_robot_id
17 ON task(assigned_robot_id);
18
19 CREATE INDEX idx_task_status
20 ON task(status);
21
22 CREATE INDEX idx_task_event_task_id
23 ON task_event(task_id);
24
25 CREATE INDEX idx_exception_log_robot_id
26 ON exception_log(robot_id);
27
28 CREATE INDEX idx_exception_log_task_id
29 ON exception_log(task_id);
30
31 CREATE INDEX idx_exception_log_order_id
32 ON exception_log(order_id);
```
